

The Future of API Documentation

API Reference

GET / api/v1/users

Parameters

- ▶ Id: string (required)
- ▶ Offset: integer

Interactive, Executable, and AI-Navigable

Testing

API Documentation

1 GET Testing

GET / api/v1/orders

POST / api/v1/create

POST / api/v1/products

8 GET POST / api/v1/products

GET https://api.v.example.com/v1.users/{id}

CREATE Params

GET Params Params

GET https://api.example.com/v1.users/{id}

CREATE https://api.example.com/v1.users/{id}

PARAMS

GET IS RESO

Response

- ▶ 1 string (required)
- ▶ 2 http: vname: value (required)
- ▶ 3. File?

Response



devm.io NEW YORK WEEK

The Problem We All Know



Card 1 of 3

Outdated guides

Docs describe last year's API



Card 2 of 3

Broken examples

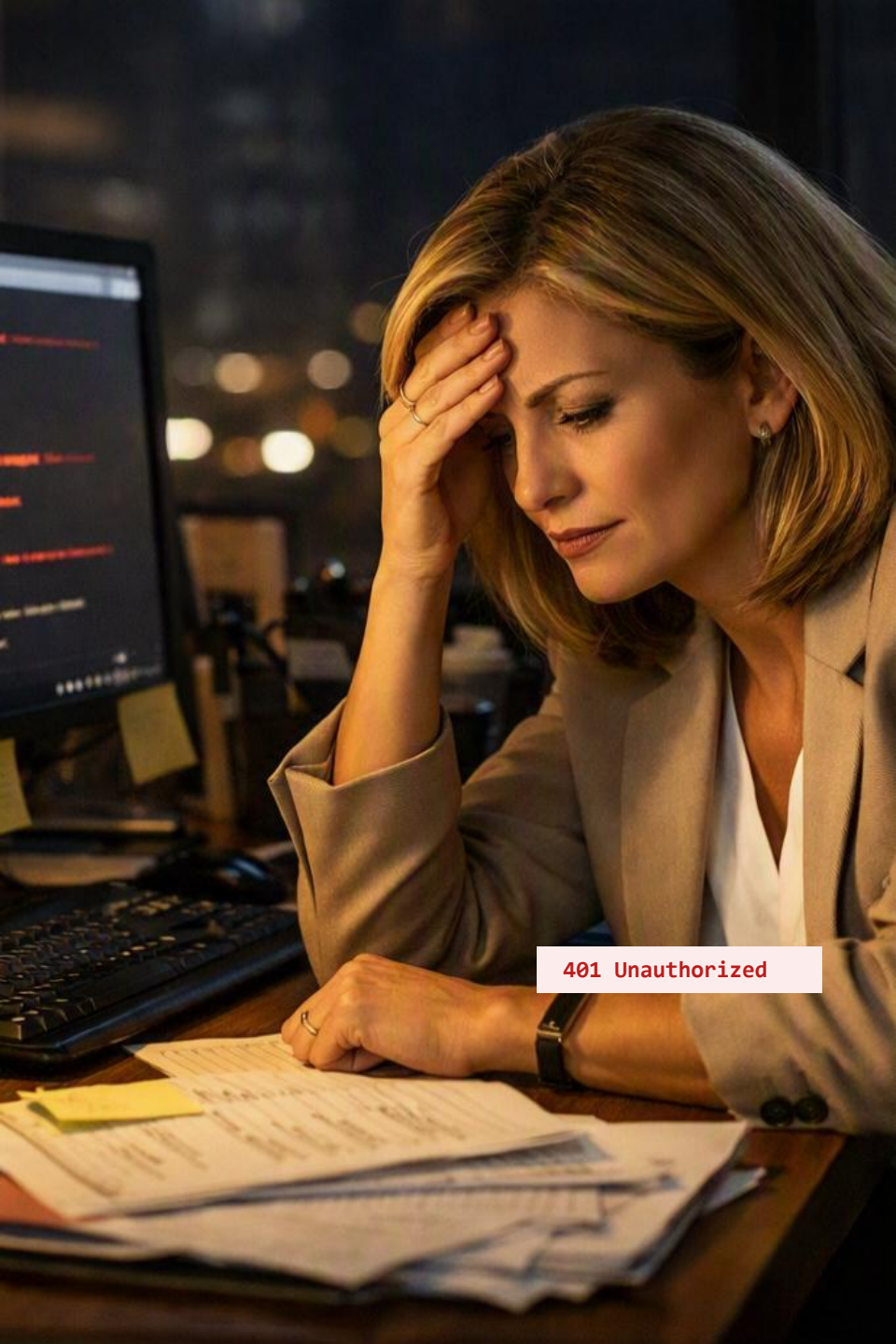
Copy. Paste. **401 Unauthorized**



Card 3 of 3

Silent drift

Code ships. Docs don't.



ACT I • 2019

Janet's Three-Day Integration

- Searches the portal: *"create payment"*
- Copies the curl example
- **401 Unauthorized**
- 40 minutes later: a Stack Overflow comment
- The API moved to OAuth. The docs still said Basic Auth.

3 days lost to documentation drift

401 Unauthorized

Why Janet's Story Still Matters

- Drift still happens; every sprint
- Most docs are still prose-first
- AI assistants now amplify the errors

**Drift isn't a nuisance.
It's a structural failure.**



“The docs weren’t wrong on purpose. They were just disconnected from reality.”

— Janet

THE THESIS

API documentation is becoming...

Interactive

Executable

AI-Navigable

Agent-Operable

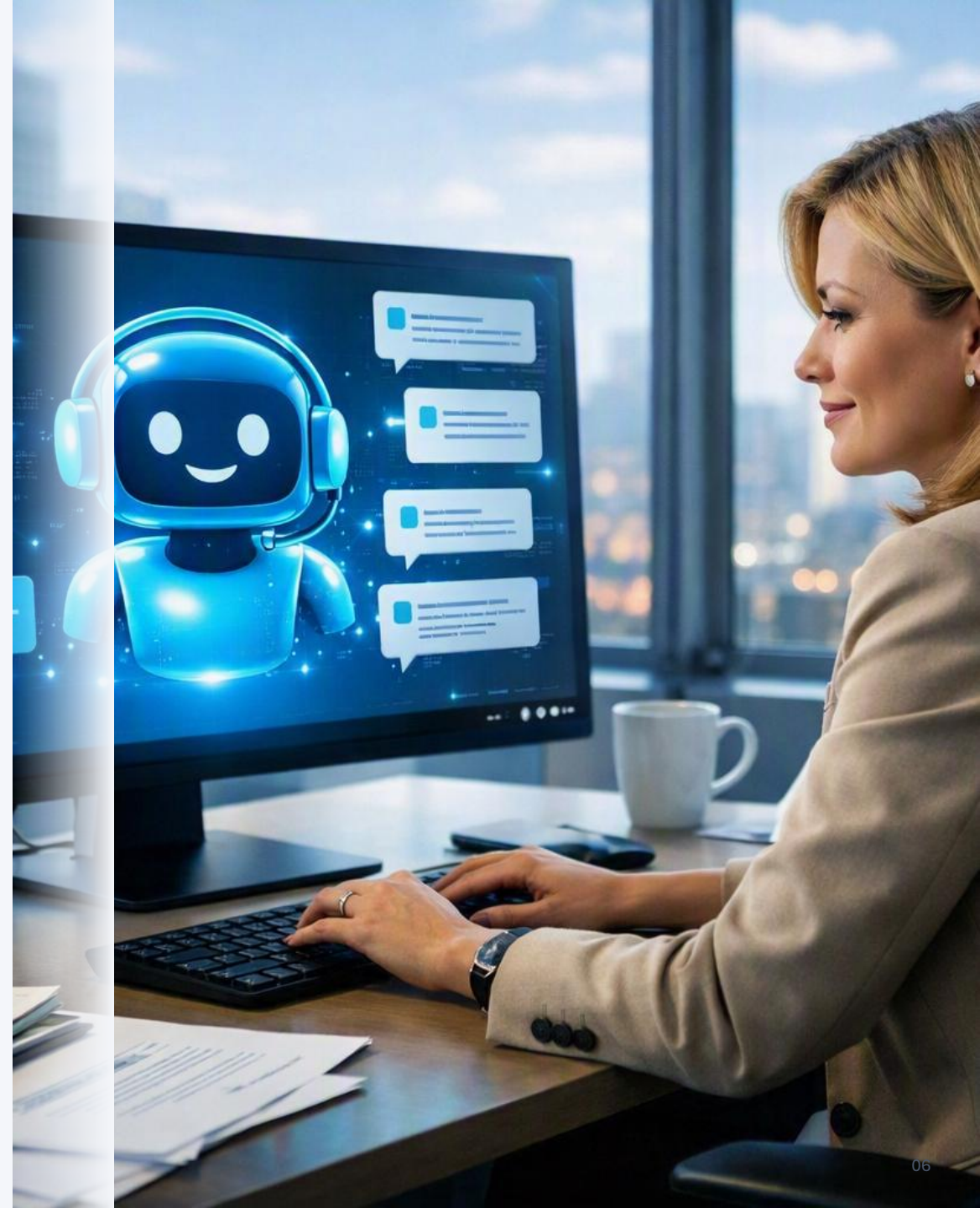
...and teams must adapt now.

02

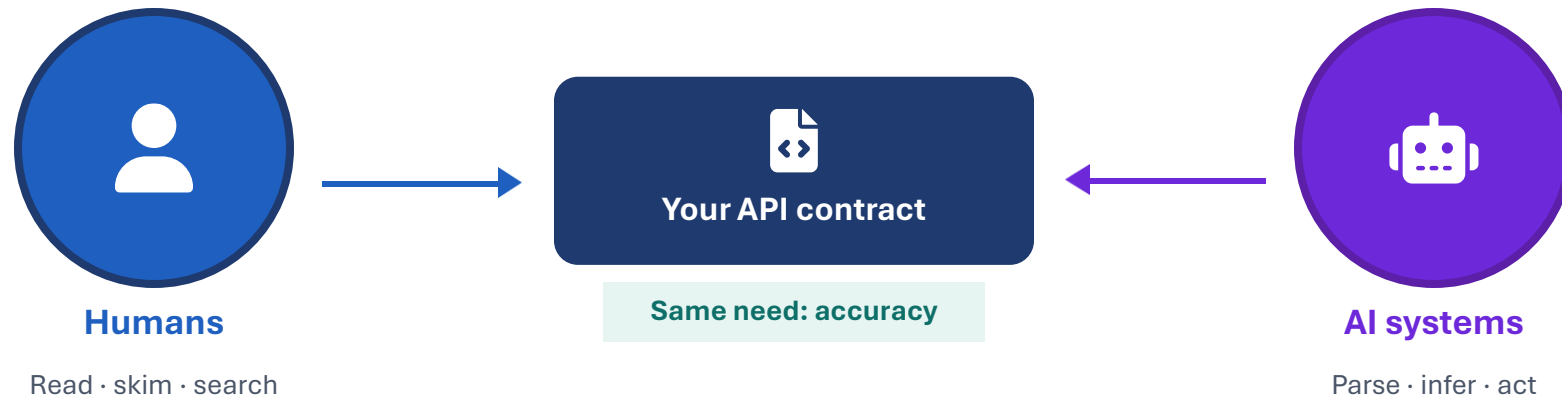
SECTION 02

The Audience Has Changed

Janet isn't the only one reading your docs anymore.



Two Audiences, One API



How do we build docs that Janet and her AI teammates can both trust?

TODAY · 2026

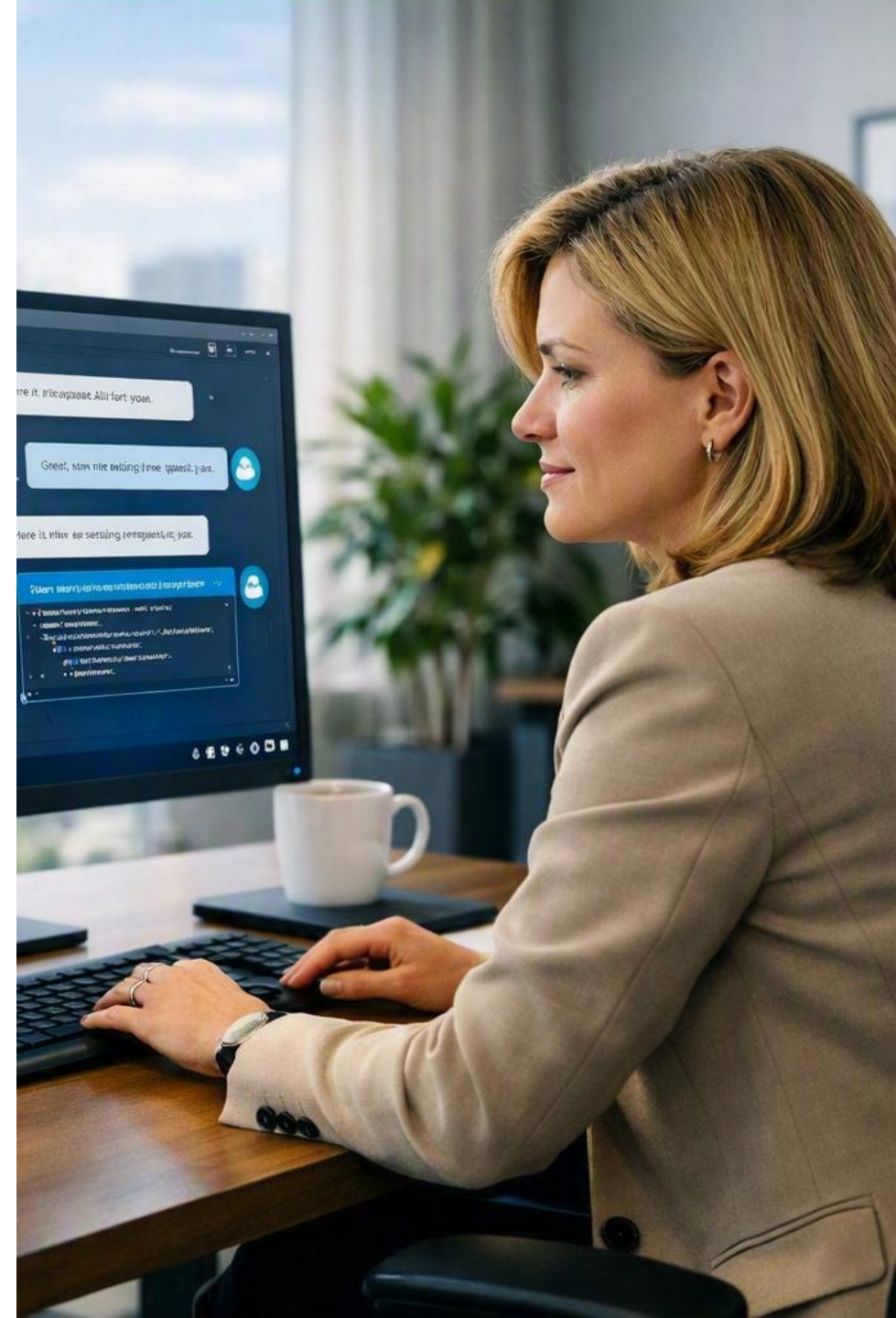
How Janet Works Today

- She doesn't search portals
- She asks her AI assistant
- It answers instantly — and confidently

⚠ New failure mode: confidently wrong code

```
auth = HTTPBasicAuth(api_key, "")
```

Generated from a semantically empty spec



Six Industry Shifts



Search

→ **Conversation**



Reading docs

→ **Executing docs**



Human

→ **Human + AI**



Portal

→ **Developer platform**



Reference

→ **Knowledge graph**



Static versions

→ **Continuous
semantic evolution**

Janet Through the Six Shifts

Shift	 Janet in 2019	 Janet today
Conversation	Keyword search in a portal	Asks her assistant in the IDE
Execution	Copies curl into a terminal	Runs the example inside the docs
Human + AI	Reads alone	Pairs with an AI assistant
Platform	Static portal pages	Sandbox, keys and SDKs in one place
Knowledge graph	Clicks through flat reference	Follows linked resources and workflows
Continuous evolution	Finds breaking changes in prod	Gets machine-readable change alerts

03

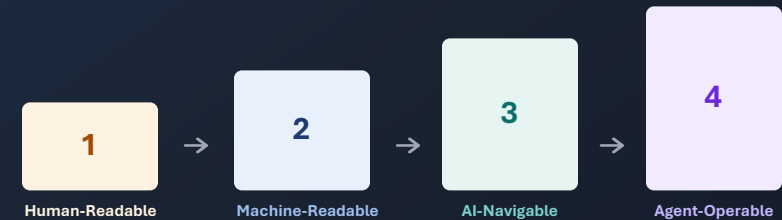
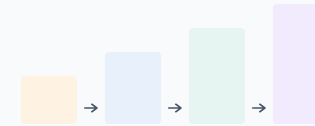
Section 03

The Four Stages of Documentation Evolution

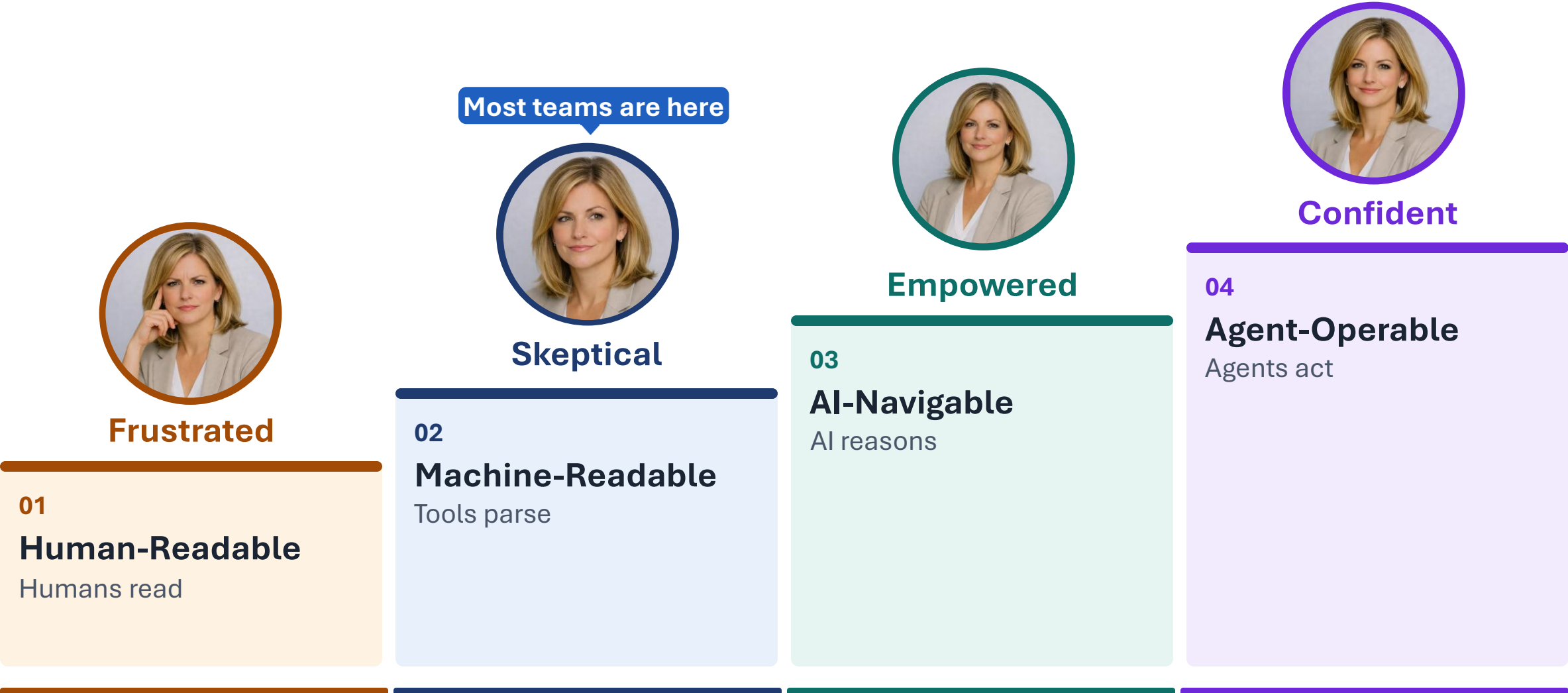
Where does your documentation sit today?



Documentation Evolution



Four Stages of Documentation Evolution



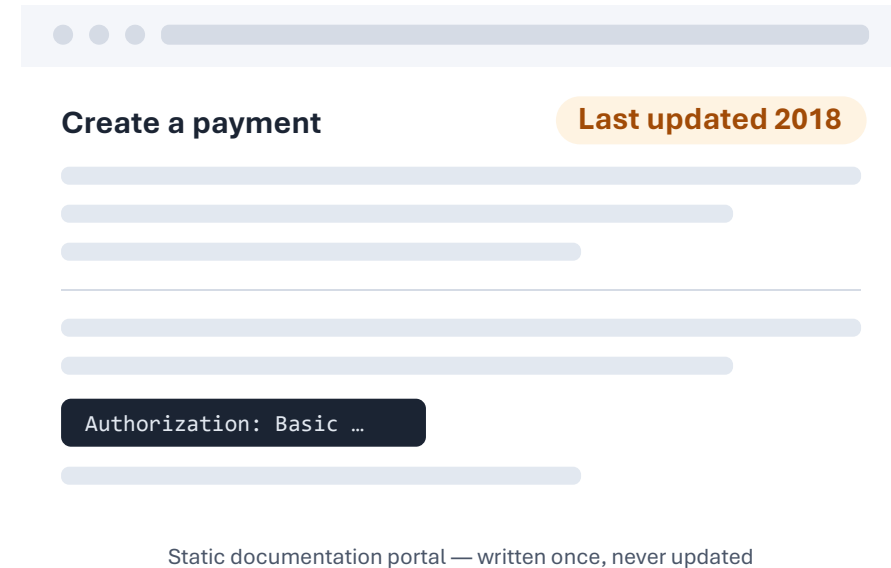
Stage 1: Human-Readable

- Prose-first portals and wikis
- Written once, updated rarely
- Drift is invisible until it breaks



"I spent more time verifying the docs than using them."

— Janet



Stage 2: Machine-Readable

- OpenAPI, AsyncAPI, GraphQL Introspection
- Structure tools can validate
- Still no meaning or intent

```
amount: — ? What does it mean?  
  type: integer  
currency:  
  type: string  
status:  
  type: string
```

✓ Valid



"I got YAML. I still had to guess the rules."

Stage 3: AI-Navigable

- Descriptions that explain intent
- Workflow metadata and error taxonomies
- Assistants stop hallucinating



“My assistant finally gives answers I don’t have to double-check.”

— Janet

How do I charge a card?

Call **createPaymentIntent** with OAuth scope **payments:write**, then **confirmPaymentIntent**. Amount is in cents.

☐ Grounded in spec

Stage 4: Agent-Operable

- MCP tools and resources
- Structured, resolvable errors
- Idempotency and scoped auth

🤖 Agent run · checkout-bot

✓ create_payment_intent

idempotent

↻ 401 → token refreshed, retried

auto-recovered

✓ confirm_payment_intent

200 OK



The 401 error told the agent exactly how to fix itself — no human in the loop.



“My newest teammates don’t read docs. They execute them.”

— Janet

Comparing the Four Stages

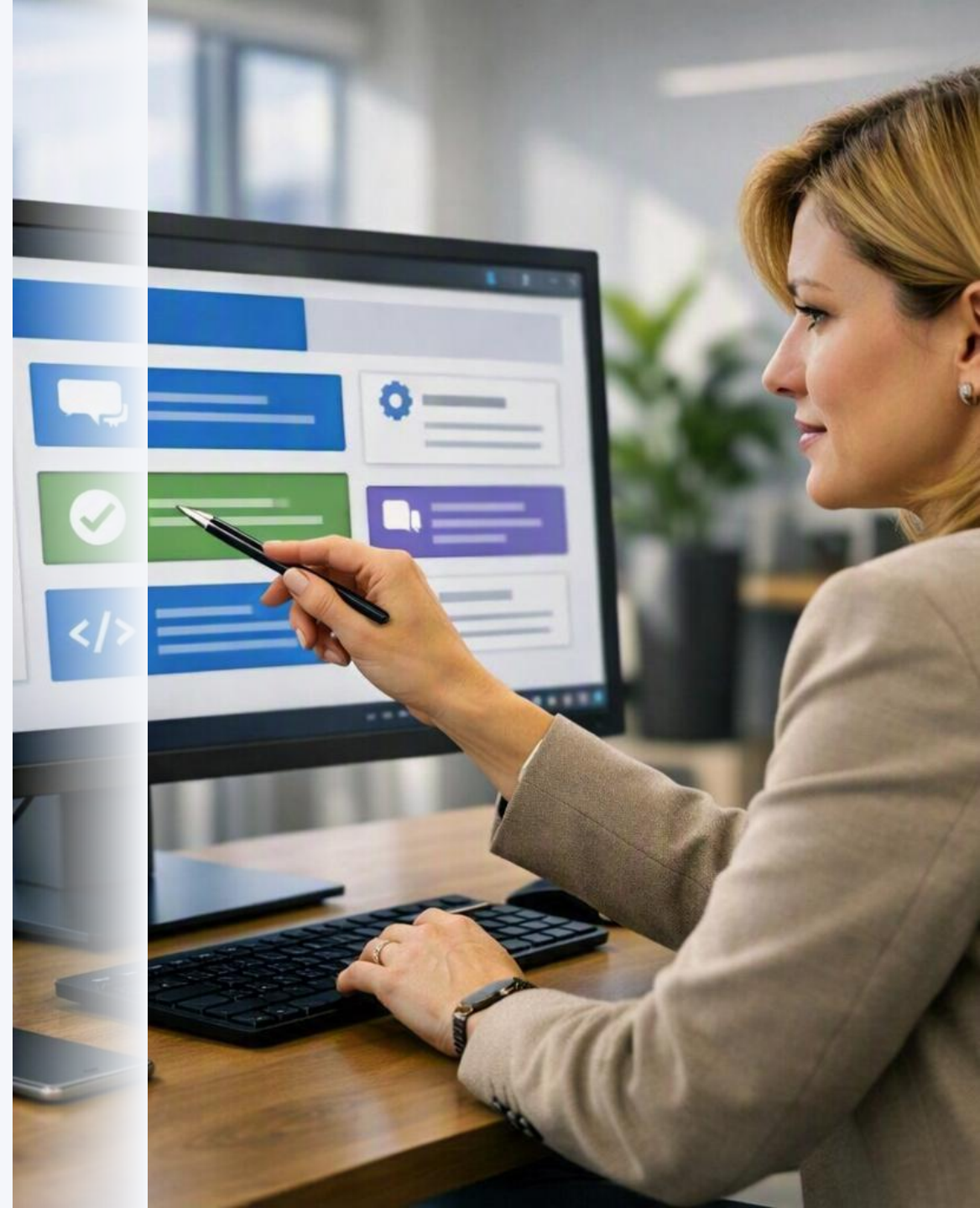
	Stage 1 Human-Readable	Stage 2 Machine-Readable	Stage 3 AI-Navigable	Stage 4 Agent-Operable
Consumer	Humans	Tools & generators	AI assistants	Autonomous agents
Format	Prose, portals	OpenAPI, AsyncAPI	Enriched specs + metadata	MCP tools + policies
Capability	Read	Validate & generate	Reason & navigate	Act safely
Failure mode	Silent drift	Valid but meaningless	Gaps → hallucination	Unsafe side effects
Janet feels	Frustrated	Skeptical	Empowered	Confident

04

SECTION 04

The Semantic Gap

Valid isn't the same as understandable.



THE SEMANTIC GAP

Structural validity $\neq$ semantic adequacy.

A spec can pass every linter and still tell a reader nothing.

✓ Validator: passes

? AI assistant: guesses

The Sparse Spec

Valid OpenAPI. Semantically empty.

✓ Validator: passes

? AI assistant: guesses

openapi/payments.yaml

```
1 paths :
2   /payment_intents :
3     post :
4       summary : Create payment intent
5       requestBody :
6         content :
7           application/json :
8             schema :
9               type : object
10            properties :
11              amount : { type : integer }
12              currency : { type : string }
13            responses :
14              '200' :
15                description : OK
```

1

1

No auth model

Basic? OAuth? Scopes? The spec is silent.

2

Dollars or cents?

No units, no minimum — integer is all you get.

3

Which currencies?

No enum — usd, eur, gbp, or anything else?

4

What next? What fails?

No workflow, no errors — just "OK".

The Enriched Spec

Same endpoint. Now it explains itself.

✓ Intent & workflow

operationId: createPaymentIntent

description: >

Reserves funds for checkout.

Step 1 of 2.

x-workflow:

name: checkout

step: 1

next: confirmPaymentIntent

x-idempotency-key: required

security:

- **oauth2:** [payments:write]

✓ Semantics & errors

amount:

type: integer

minimum: 50

description: Minor units (cents)

currency:

type: string

enum: [usd, eur, gbp]

responses:

'201': PaymentIntent

'401': AuthError # *refresh token*

'422': ValidationError

①

Auth answered

OAuth 2.0 · scope payments:write

②

Units answered

Minor units (cents) · minimum 50

③

Currencies answered

Enum: usd, eur, gbp

④

What next · What fails — answered

Workflow step 1→2 · 401 refresh
· 422 validation

Three Days vs. Thirty Minutes



Sparse spec

3 days

Guessing auth, units and errors



Enriched spec

30 minutes

The spec answers before she asks

AI assistants are only as good as the semantics you give them.

The Six Semantic Investments

The enrichment checklist that closes the semantic gap

1



Operation intent

Why this call exists

2



Workflow sequencing

What comes before and after

3



Parameter semantics

Units, ranges, enums, formats

4



Structured error taxonomy

What failed and how to fix it

5



Resource relationships

How objects connect

6



Machine-readable changelogs

What changed — for tools too

Janet's Workflow, Friction Removed

Investment	Without	With
1 Intent	✗ Guesses which endpoint	✓ Finds createPaymentIntent first try
2 Workflow	✗ Guesses the call order	✓ Follows step 1 → step 2
3 Parameters	✗ Dollars or cents?	✓ Minor units, minimum 50
4 Errors	✗ Hunts for validation rules	✓ 422 names the field and the fix
5 Relationships	✗ Hand-maps objects	✓ Follows links to Customer, Refund
6 Changelogs	✗ Learns of OAuth switch in prod	✓ Alerted before the change ships



"If I'd had workflow metadata, I wouldn't have guessed the sequence. If I'd had structured errors, I wouldn't have hunted for rules." — Janet

05

Section 05

Drift as a Security Problem

"What you don't document, you can't defend."

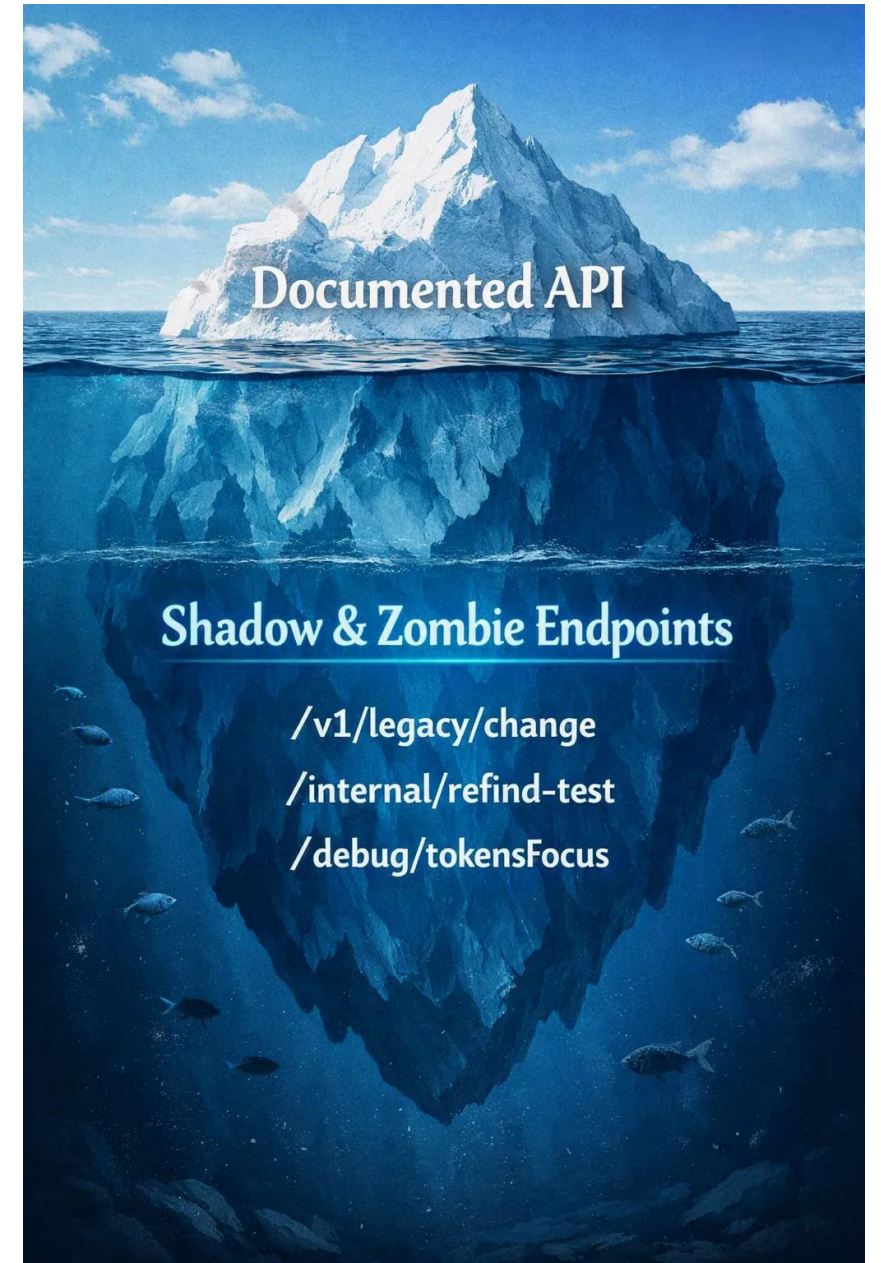


Drift Is No Longer Just a DX Problem

- Shadow APIs are undocumented attack surfaces
- Wrong docs means wrong authorization assumptions
- Unknown endpoints never get reviewed



**Attackers don't read your portal;
they read your traffic.**



Two Dangerous Drift Modes



Undocumented Endpoints

- Exist in code, not in the spec
- Never reviewed, never tested
- Found by attackers first

 **Attack surface**



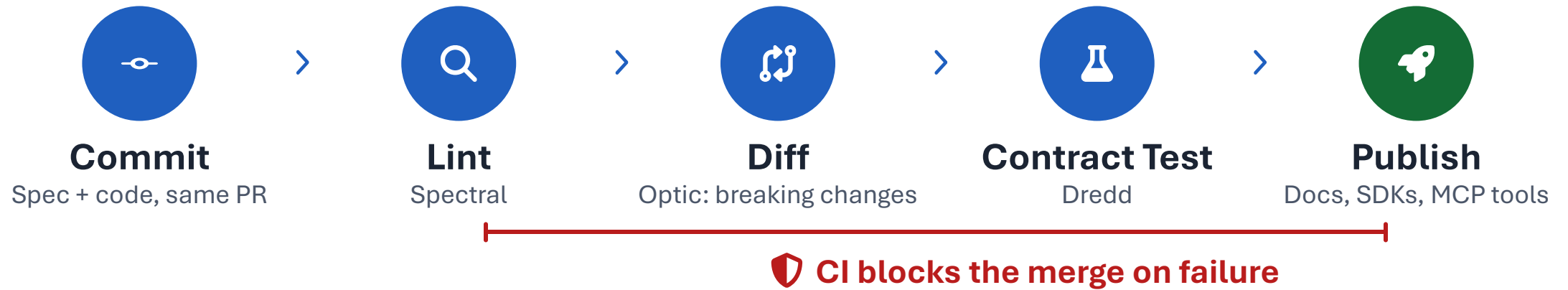
Documented-but-Wrong

- Spec says one thing, code another
- Clients build on false assumptions
- Auth and validation gaps

 **Security misconfiguration**

“ Janet hit the second one — **both are security risks, not just DX problems.**

Git-Native Documentation Pipelines



"Docs are production code."



ACT I, REVISITED

Janet's Security Blind Spot

Docs said: Basic Auth

≠

API required: OAuth 2.0

Her 401 wasn't just a DX issue. It was a security misconfiguration.

"I wasn't just slowed down. I was integrating blind."

— Janet

06

SECTION 06

Completing the API Surface

Endpoints are only part of the story.



Endpoints

OpenAPI



Events

AsyncAPI



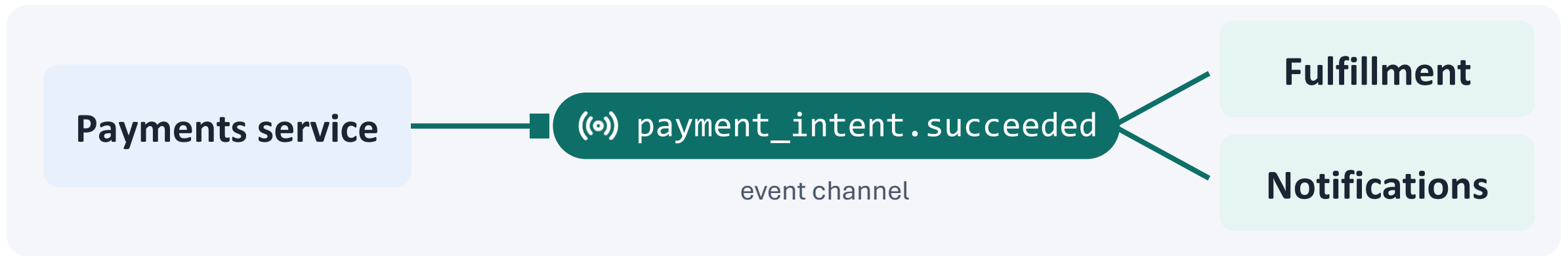
Relationships

Knowledge Graph



Tools

MCP



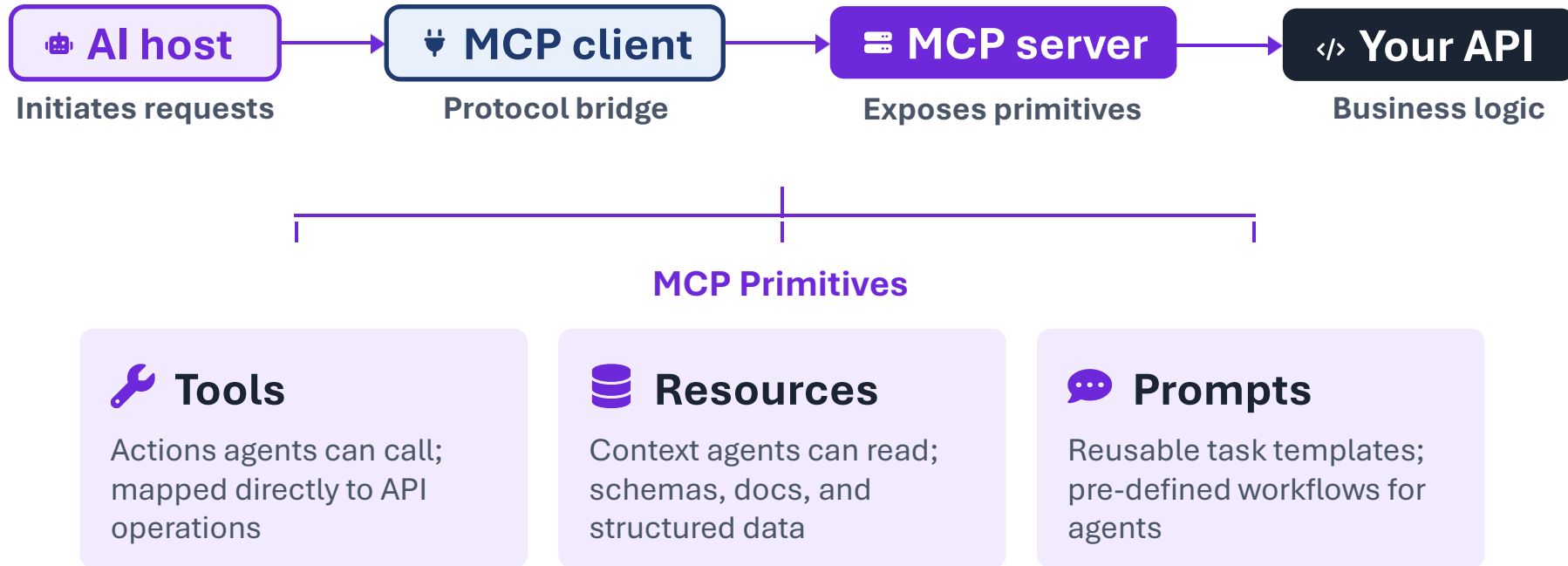
Why events need contracts

- Events need contracts too
- Channels, payloads, delivery semantics
- Same tooling: lint, diff, generate

AsyncAPI 3.0 `asyncapi/events.yaml`

```
asyncapi: 3.0.0
channels:
  paymentSucceeded:
    address: payment_intent.succeeded
    messages:
      paid:
        $ref: '#/.../PaymentSucceeded'
```

MCP: Making APIs Agent-Callable

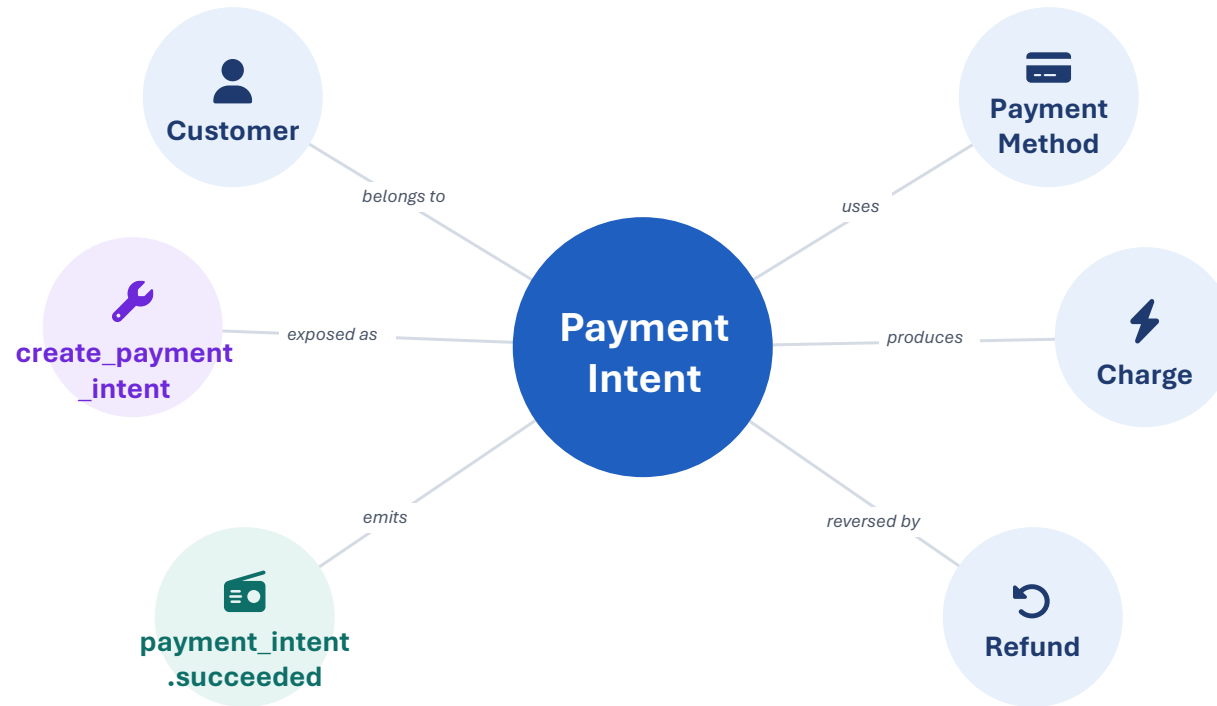


⚡ Your API becomes a surface agents can discover and call.

Knowledge Graphs for Reasoning

Techniques

- HAL & JSON:API links
- ALPS semantic profiles
- OpenAPI links / x-links
- GraphQL introspection



From flat reference to navigable system.

Janet's Full API Surface

</>

Endpoints

REST operations — the core contract

OpenAPI

“|”

Events

Async channels — what happens next

AsyncAPI

🔗

Relationships

Linked resources — how objects connect

Knowledge graph

🔧

Tools

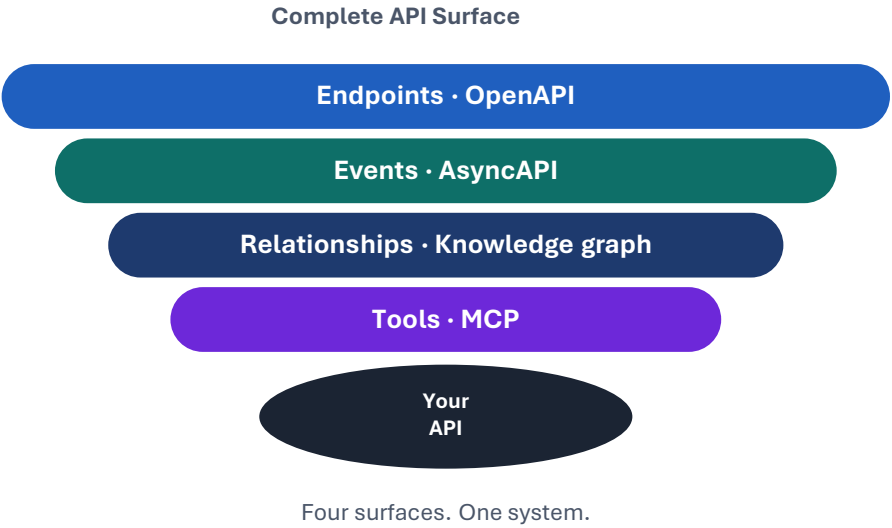
Agent-callable actions — operable surface

MCP



"My API isn't just endpoints. It's a system."

— Janet

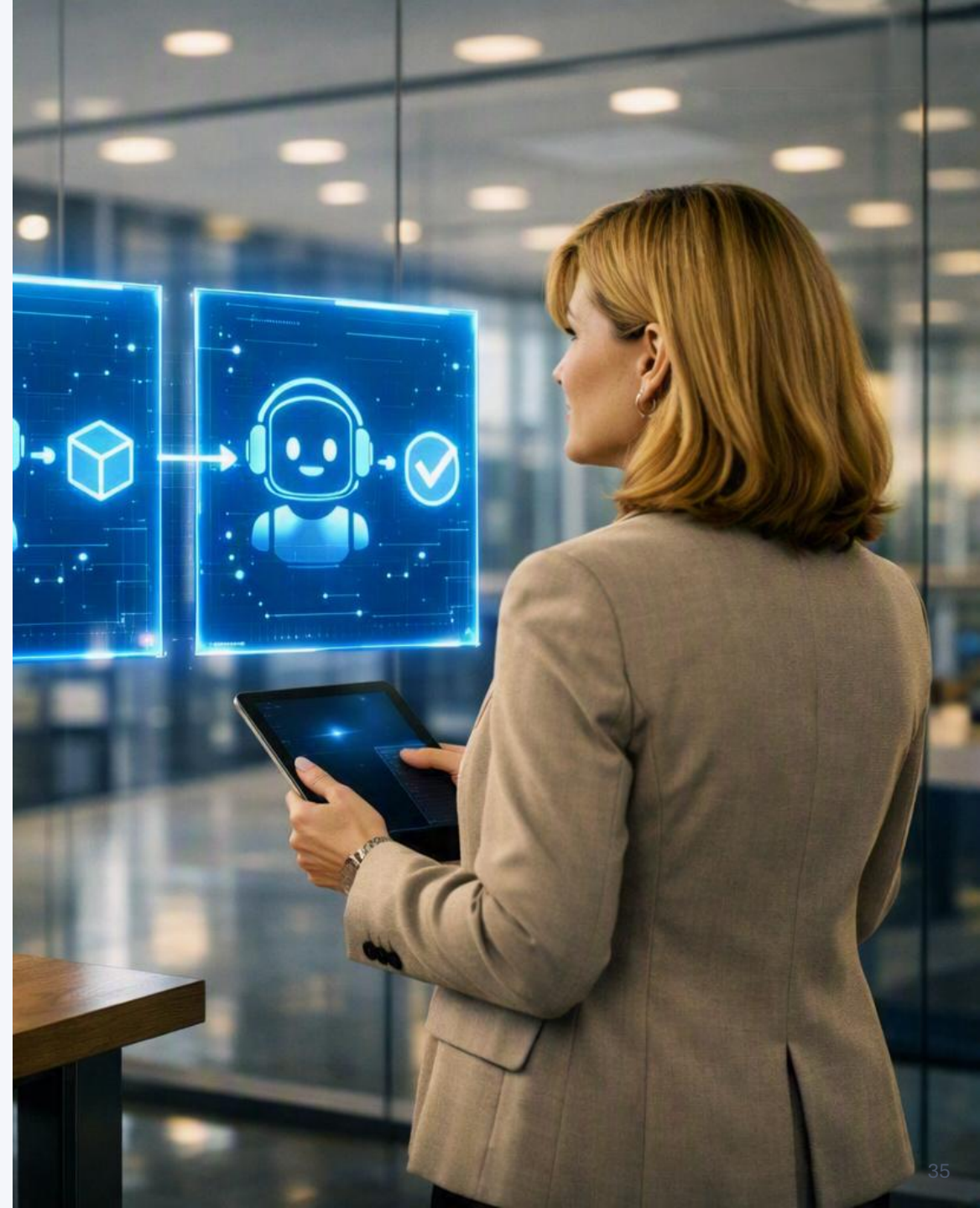


07

SECTION 07

Agent-Operable APIs

When the caller has no human in the loop.



What Agents Need From Your API



Idempotency

Safe to retry



Scope-limited auth

Least privilege by default



Side-effect classification

Read, write, or destructive?



Rate-limit semantics

Limits agents can plan around



Structured error resolution

Errors that say how to recover

Near Future

From Integrator to Orchestrator

Integrator



Orchestrator

Integrator

- Reads docs, writes glue code
- Debugs one call at a time

Orchestrator

- Sets goals, scopes, guardrails
- Reviews what agents did — and why

"I'm not integrating APIs anymore. I'm supervising the agents that do."

— Janet



An Agent-Operable Tool Definition

The same PaymentIntent, as an MCP tool

```

1  {
2  ①  "name" : "create_payment_intent" ,
3      "description" : "Step 1 of 2: reserve checkout funds" ,
4      "inputSchema" : {
5          "type" : "object" ,
6          "properties" : {
7  ②      "amount" : { "type" : "integer" , "minimum" : 50 },
8          "currency" : { "enum" : [ "usd" , "eur" , "gbp" ] },
9  ③      "idempotency_key" : { "type" : "string" }
10     },
11     "required" : [ "amount" , "currency" , "idempotency_key" ]
12 },
13 ④  "annotations" : {
14      "destructiveHint" : false ,
15      "idempotentHint" : true
16  }
17 }
```

① Intent + sequence

Names the operation and declares it as step 1 of 2 — no guessing required

② Constraints agents can't misread

Minimum 50, enum of valid currencies — no ambiguity about units or values

③ Safe retries

Required idempotency key — agents can retry without creating duplicate charges

④ Side effects declared up front

Not destructive · idempotent — the client knows before it calls

The Next Five Years

Agents will consume your APIs at scale.

Documentation becomes infrastructure.



Human-Readable



Machine-Readable



AI-Navigable



Agent-Operable

08

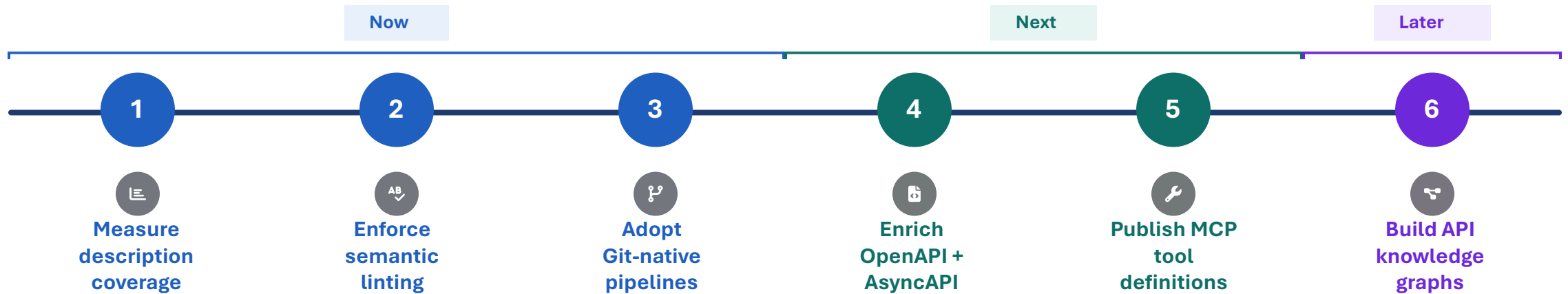
SECTION 08

Practical Roadmap

Six steps you can start Monday.



The Roadmap: What Teams Must Do Now

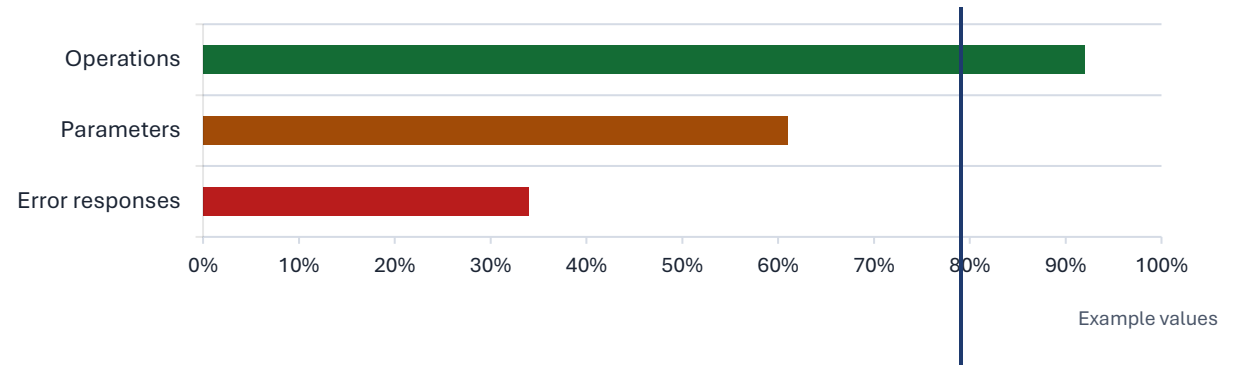


 Principle: treat documentation as infrastructure

Measure Description Coverage

- Track described operations, parameters, and schemas
- Treat semantic completions as a quality metric
- Baseline first, then set a target

Description Coverage by Category



**Target
80%**



"You can't fix what you don't measure." — Janet

Enforce Semantic Linting

- Spectral rulesets in every pull request
- Require descriptions, examples, error schemas
- Custom rules for your x-* extensions



"The linter catches the gaps before my assistant does." — Janet

Spectral Rule — operation-intent

```
rules :  
  operation-intent:  
    description: Operations state intent  
    given: $.paths[*][*]  
    severity: error  
    then :  
      field: description  
      function: truthful
```

× 3 operations missing intent

Adopt Git-Native Pipelines

- Specs live beside the code they describe
- CI validates, diffs, and contract-tests
- Merge is blocked when docs drift



“If the docs can’t drift, I can trust them.” — Janet

🔗 payments-api/

```
├─src/
├─openapi/payments.yaml ← OpenAPI spec
├─asyncapi/events.yaml ← AsyncAPI spec
├─mcp/tools.json ← MCP tools
└─.github/workflows/docs.yml
```

🛡️ CI blocks the merge when docs drift from code

Build Enriched OpenAPI + AsyncAPI

- Add intent, workflow, and constraint metadata
- Structured error schemas for every failure
- Consider Azazzo for multi-step workflows

One PR. Three answers.

openapi/payments.yaml – enriched diff

```
amount:
  type: integer
+ minimum: 50
+ description: Minor units (cents)
...
currency:
  type: string
+ enum: [usd, eur, gbp]
...
responses:
  '201': PaymentIntent
+ '401': AuthError # refresh token
+ '422': ValidationError
```



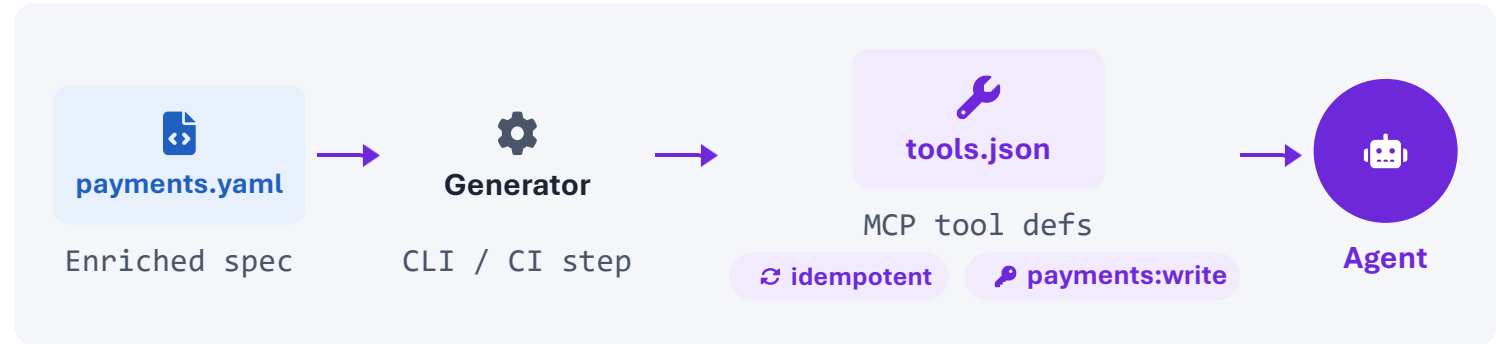
“This is the difference between three days and thirty minutes.” — Janet

Gaps closed: Units & constraints Currency enum Error schemas

Publish MCP Tool Definitions

- Generate tools from the enriched spec
- Declare side effects and idempotency
- Scope auth to least privilege

Generation Flow



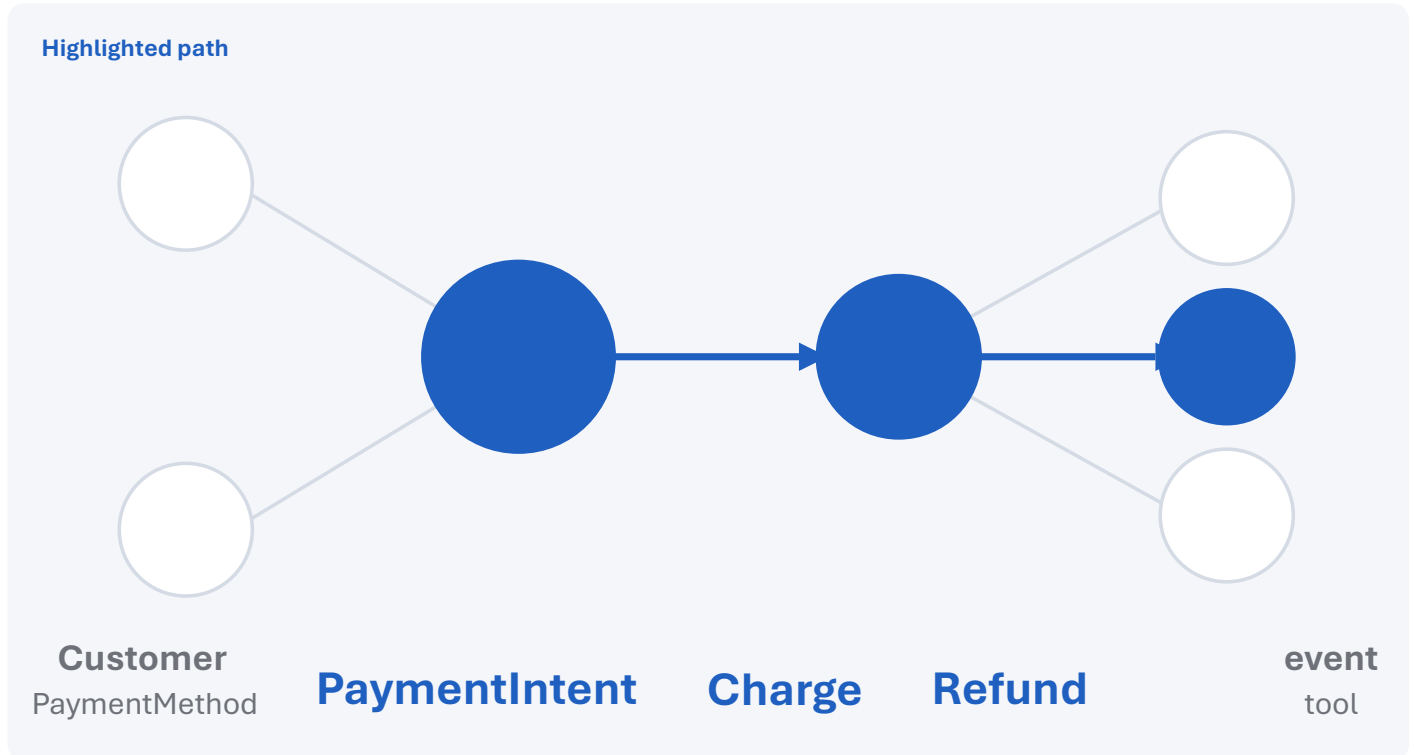
“Now my agents call the API the way I would; carefully”

— Janet

Construct API Knowledge Graphs

- Link resources, events, and tools
- Publish relationships machines can traverse
- Power semantic search and AI reasoning

🔍 *refund a partial payment*



“I ask a question. The graph knows the path.”

— Janet

.....

Janet's Future Workflow

- **09:00** Asks: *"Add card payments to checkout"*
- **09:03** Semantic search finds the checkout workflow
- **09:06** Runs the example live in the docs
- **09:11** Agent scaffolds the client with idempotency keys
- **09:18** Contract tests pass — ✓ 200 OK

Minutes, not days.



09

SECTION 09

Closing

Back to Janet.



ACT III • 2026

Same API. Same Task. Different World.

- Enriched OpenAPI stated the auth model
- Executable examples ran the first time
- Semantic search found the workflow
- MCP tools let her agent finish

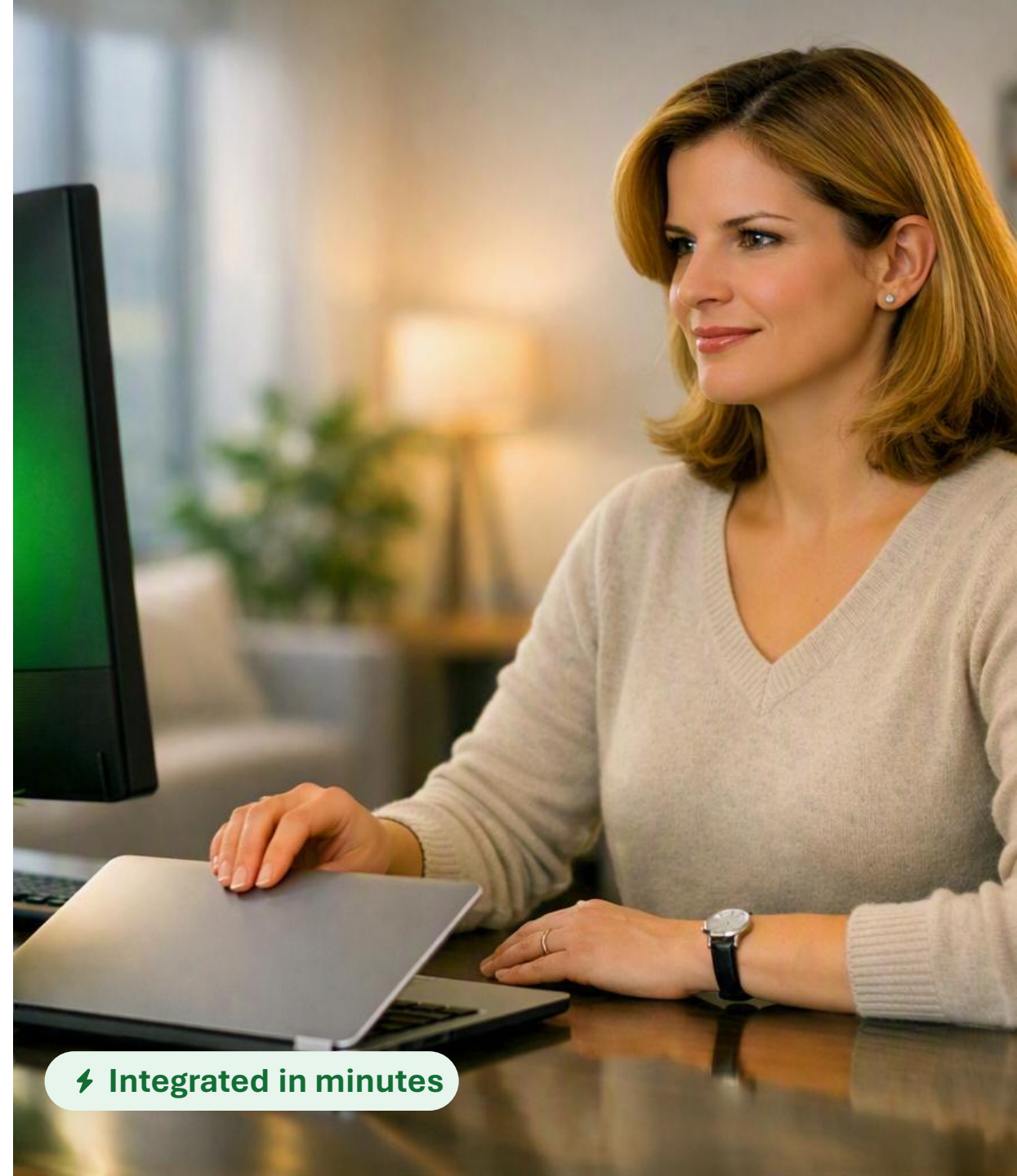
This is the world we can build — if we choose to.

2019: Three Days

Guessing auth, units & errors

Now: Minutes

The spec answered before she asked



⚡ Integrated in minutes

Your Next Step

Build documentation for humans and agents.

Treat documentation as infrastructure.



Measure coverage this week



Add one semantic lint rule

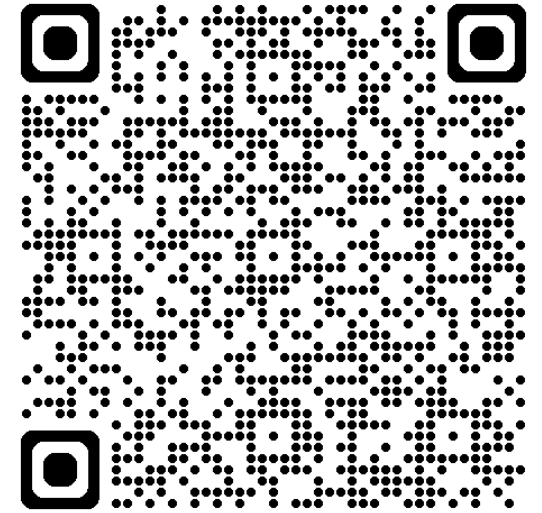


Publish one MCP tool



Thank you

Questions?



Chad Green

Senior Software Architect, Jasper Engines & Transmissions

Microsoft MVP · @ChadGreen

<https://chadgreen.com>